

FLIGHT SOFTWARE V1 — SYSTEM REPORT

How the OBC↔ADCS Flight Software Stack Works

Architecture, the position-command data flow, the hardware/simulation abstraction, testing infrastructure, and where the project stands.

Branch: `task-integration`

Mission: November 1st Balloon Launch

Scope: OBC (Linux) ↔ v-bus/HAL ↔ CSP ↔ ADCS (FreeRTOS)

§ Contents

1 System Architecture	6 Testing Infrastructure
2 The Position-Command Flow	7 Notable Bugs Found & Fixed
3 Hardware/Simulation Abstraction (v-bus)	8 Power — Current Capability & Potential
4 RTOS Task Reference	9 Next Steps
5 CSP Transport Layer	10 Appendix — File Map & Commands

1 System Architecture

The stack is two independent OS processes standing in for two independent physical boards, talking over a bus abstraction that can be backed by either a Unix domain socket (development) or a real SPI peripheral (flight hardware) without either process's own code changing:

- **OBC (On-Board Computer)** — `apps/obc/src/main.c`. A plain Linux/POSIX process. Owns the ground-facing role: it is the SPI bus *master*, and today it periodically issues attitude commands and listens for telemetry.
- **ADCS (Attitude Determination & Control)** — `apps/adcs/src/main.c`. A FreeRTOS application, running its scheduler on top of FreeRTOS's POSIX simulation port (real pthreads standing in for hardware interrupts/ticks). It is the SPI bus *slave*, and hosts six tasks.
- **v-bus** — `platform/include/v_bus.h`. The bus abstraction both processes' CSP layer talks to. Never SIM- or HW-specific above this header (see §3).
- **CSP (CubeSat Space Protocol)** — vendored at `libs/libcsp/` (git submodule), wired up by `shared/csp/`. Provides addressing, port-based multiplexing, packet framing and CRC on top of v-bus.

Process/Thread Topology

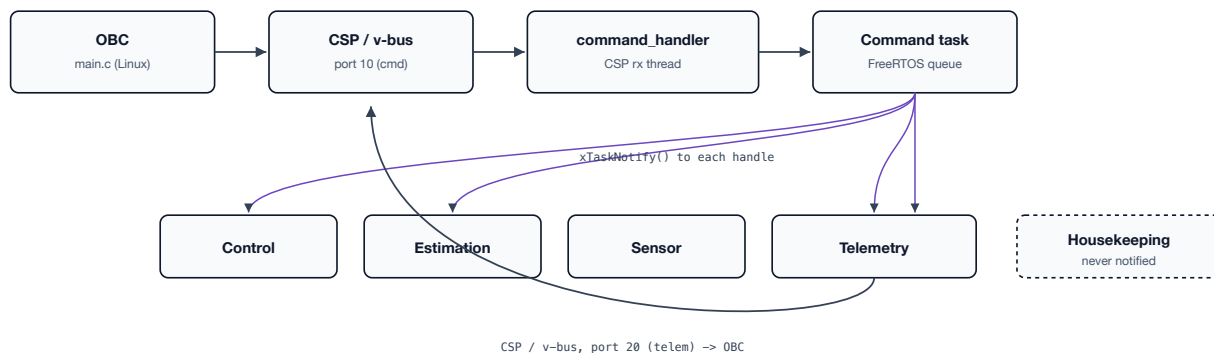
Each process runs several concurrent threads. This matters a lot for §7 — most of the real bugs found this session were exactly at the seams between these contexts.

Process	Thread	Role
OBC	main thread	Sends a position command every <code>OBC_POSITION_PERIOD_SEC</code> (default 30s)
	telemetry rx thread	Blocks on CSP <code>accept / read</code> for ADCS telemetry replies
ADCS	CSP RX thread	Spawned inside <code>csp_if_spi_init()</code> ; pulls raw bytes off v-bus into CSP's queue

	CSP router thread	Spawned by <code>csp_network_init()</code> ; pumps <code>csp_route_work()</code> to deliver queued packets to bound sockets
	command_handler thread	Plain pthread; CSP <code>accept / read</code> loop on the command port, decodes and hands off to the Command task's queue
	6× FreeRTOS tasks	Command, Control, Estimation, Housekeeping, Sensor, Telemetry — see §4

2 The Position-Command Flow

This is the one thing proven to work fully end to end, and it is the best concrete example of how every layer above cooperates. Every 30 seconds (configurable), the OBC tells the ADCS to move to a new 1-D position; that single command fans out to every ADCS task except Housekeeping, and Telemetry reports the outcome back.



Step by step

- OBC builds and sends the command.** `apps/obc/src/main.c` increments a target position (wrapping through a small fixed range), fills a `position_command_t` (`shared/csp/csp_commands.h`) into a CSP packet, and sends it via `csp_connect / csp_buffer_get / csp_send / csp_close` to ADCS address 2, port 10 (`ADCS_CMD_PORT`).
- CSP carries it across v-bus.** The packet is framed (`csp_id_prepend`) and handed to the transport's `send()` — which, in SIM mode, is a `write()` on the Unix domain socket at `/tmp/v_bus.sock` .
- ADCS receives and routes it.** A dedicated RX pthread (started inside `csp_if_spi_init`) blocks on `v_bus_receive()` , reconstructs the packet, and hands it to CSP's internal queue. A second pthread (the "router thread", started by `csp_network_init`) continuously calls `csp_route_work()` to deliver it to whichever socket is bound to port 10.

4. **command_handler.c decodes it.** Its own dedicated pthread is bound/listening on port 10; it `csp_read()`s the packet, decodes the `position_command_t` payload, and calls `command_task_send()`.
5. **Command task fans out.** `command_task.c` dequeues the message and calls `xTaskNotify()` on the task handles of Control, Estimation, Sensor, and Telemetry — *not* Housekeeping. Each of those four tasks does a non-blocking `xTaskNotifyWait()` check at the top of its normal periodic loop and, on a hit, logs that it was activated.
6. **Telemetry closes the loop.** On its own notification, `telemetry_task.c` additionally builds a `position_telemetry_t` and sends it back to OBC address 1, port 20 (`ADCS_TELEM_PORT`) — the same `csp_connect` / `csp_send` / `csp_close` pattern, in reverse.
7. **OBC reports the result.** Its telemetry rx thread reads the reply and prints the ADCS-reported position, proving the full round trip.

Why `command_task_send()` uses `xQueueSendFromISR()`, not `xQueueSend()`

`command_handler.c`'s rx thread is a plain `pthread_create()` thread, not a FreeRTOS task. FreeRTOS's ordinary queue API assumes the caller is a scheduled task with a valid internal task-control-block; called from a foreign thread it silently hung *after* the data had already been copied — which is why exactly the first command always got through and every one after it vanished. The `FromISR` variant is the documented, correct way to feed a FreeRTOS queue from any non-task context. See §7.

3 Hardware/Simulation Abstraction (v-bus)

The whole point of v-bus is that nothing above the platform layer should care whether it is talking to a Unix socket or a real SPI peripheral. That was *mostly* true already — a full search of apps/ turns up zero direct references to v-bus, sockets, or SPI — but one seam remained: `shared/csp/csp_network.c`, the single place that wires v-bus into CSP, hardcoded `#include "../platform/sim/include/v_bus.h"`. That one line meant the CSP layer itself could never be pointed at the hardware backend without editing source.

The fix

Before	After
<code>platform/sim/include/v_bus.h</code> — the interface lived inside the SIM backend's own directory. <code>shared/csp/csp_network.c</code> included it by a hardcoded relative path into <code>sim/</code> .	<code>platform/include/v_bus.h</code> — one canonical interface, medium-agnostic, in the directory <code>platform/CMakeLists.txt</code> already exposed publicly to every consumer regardless of mode.

platform/real/ was completely empty — `DHW_MODE=ON` failed at CMake *configure* time (Cannot find source file: `real/spi/spi_driver.c`).

shared/csp/csp_network.c now does `#include "v_bus.h"` — no path, no mode knowledge.

platform/real/spi/spi_driver.c now exists, implements the identical `VBus_t` contract, and `DHW_MODE=ON` configures and builds cleanly.

How the switch actually happens

```
platform/CMakeLists.txt

if (HW_MODE)
    add_library(platform STATIC real/spi/spi_driver.c)
else()
    add_library(platform STATIC sim/spi/v_bus.c)
endif()

target_include_directories(platform PUBLIC
    ${CMAKE_SOURCE_DIR}/platform/include      # <- v_bus.h lives here, always
    ${CMAKE_CURRENT_SOURCE_DIR}/sim/include   # or real/include, depending on HW_MODE
)
```

One CMake option selects both the implementation file and the include path. Everything above `platform` links against the same public interface either way.

Honesty check: what "HW_MODE=ON builds" does and doesn't mean

`platform/real/spi/spi_driver.c` compiles and links today, and the abstraction is genuinely proven — but its body is an explicit, clearly-labeled stub (returns `V_BUS_ERROR`, prints why) rather than working hardware code, because this repository has no STM32 HAL vendored and no ARM cross-compilation toolchain wired into the build. The file is written with the real STM32 HAL call shape already sketched behind a `STM32_HAL_AVAILABLE` guard, so a future contributor with the actual HAL headers has a concrete, minimal amount of work left rather than a blank page. This is real, necessary future work — see §9.

4 RTOS Task Reference

Task	Priority	Period	Reacts to position command?	What it does today
Command	2	event-driven (queue)	—	Dequeues <code>CommandMessage_t</code> , fans out via <code>xTaskNotify</code>

Control	4 (highest)	50ms	Yes	Logs the target position it was told to slew toward
Estimation	3	100ms	Yes	Logs that it would update the attitude estimate
Sensor	3	100ms	Yes	Logs that it would sample sensors
Telemetry	1	1000ms	Yes	Logs, then sends <code>position_telemetry_t</code> back to OBC over CSP
Housekeeping	1 (lowest)	1000ms	No — by design	Idle loop only; not wired to the command path at all

All six are created with `xTaskCreateStatic` (static allocation — `configSUPPORT_STATIC_ALLOCATION` is on), which is why `apps/adcs/src/freertos_hooks.c` exists: FreeRTOS requires the application to supply `vApplicationGetIdleTaskMemory` / `vApplicationGetTimerTaskMemory` for the Idle and Timer tasks under static allocation, and nothing provided them before this session.

Every task body beyond "notice the notification and log it" is intentionally a stub — no control math, no real sensor reads, no Kalman filter. That is deliberate scope for the current milestone (prove the *flow*, not the science) and is exactly what §9 picks up.

5 CSP Transport Layer

CSP (CubeSat Space Protocol, `libcsp`) is vendored as a git submodule and provides addressing, port multiplexing, and packet framing. This build has `CSP_USE_RTABLE` off — with exactly one interface per node, each node simply marks its v-bus interface `is_default`, which is sufficient routing.

Node	CSP address
OBC	1
ADCS	2

Port	Direction	Subsystem	Status
10	OBC → MCU	ADCS Command	WIRED
11	OBC → MCU	EPS Command	DOCUMENTED ONLY
12	OBC → MCU	COMMS Command	DOCUMENTED ONLY
20	MCU → OBC	ADCS Telemetry	WIRED

21	MCU → OBC	EPS Telemetry	DOCUMENTED ONLY
22	MCU → OBC	COMMS Telemetry	DOCUMENTED ONLY
30	Both	Ground Link	DOCUMENTED ONLY

(Port table per `docs/architecture.md`; ports 10/20 are the only two actually implemented in code today, in `shared/csp/csp_commands.h`.)

One config value had to change from libcsp's own default: `CSP_PORT_MAX_BIND` defaults to 16, which silently rejected `csp_bind()` on port 20 (telemetry) while port 10 (commands) worked fine — see §7. It is now set to 31 in the top-level `CMakeLists.txt`, covering the full documented port range.

6 Testing Infrastructure

Two integration tests, both registered with CTest and gated by `BUILD_TESTS` (SIM-only):

`v_bus_test`

Links platform directly and forks two processes that call the real `v_bus_initialize / send / receive` functions in the OBC/ADCS roles, exchanging a known message both directions. A 5-second `alarm()` per process fails loudly instead of hanging. Runtime: well under a second.

`position_command_test`

The logic under test here is spread across a full FreeRTOS scheduler, not a couple of synchronous functions, so this test instead spawns the *real compiled* `obc_sim / adcs_sim` binaries as child processes (paths injected via `$<TARGET_FILE:...>` so it always tests what was actually built), overrides `OBC_POSITION_PERIOD_SEC` to 2 seconds so a ~13-second capture window covers several full cycles, and greps captured stdout for every marker in the §2 flow. Critically, it requires each marker to appear **at least 4 times**, not merely once — an earlier version only checked presence, which would have missed the exact "works once then hangs" bug in §7 entirely. It also asserts ADCS's received-command count keeps pace with OBC's sent-command count, and that Housekeeping never appears in the log.

```
cmake --build build --target position_command_test
ctest --test-dir build -R position_command_test --output-on-failure
```

Both tests were sabotage-verified, not just run once

For every bug fixed this session, the fix was confirmed by deliberately reintroducing the bug, re-running the relevant test to watch it fail with a specific, correct diagnosis, then reverting. This was done for the `SOCK_SEQPACKET` regression, the `CSP_PORT_MAX_BIND` silent drop, and the `xQueueSend` foreign-thread hang.

7 Notable Bugs Found & Fixed

A condensed record — useful context for why parts of the code look the way they do.

macOS/libcsp portability (submodule patches, all `__APPLE__`-guarded)

- `__noinit` section attribute used ELF syntax invalid on Mach-O.
- macOS has no working unnamed POSIX semaphores (`sem_init` returns `ENOSYS`) — replaced with a mutex/condvar binary semaphore on Apple only.
- No `pthread_condattr_setclock()` on Apple — timed waits switched to match Apple's implicit `CLOCK_REALTIME` behavior.
- No `<endian.h>` on Apple — shimmed via `OSByteOrder.h`, needed by 17 core CSP files.
- Homebrew's non-standard install prefix (`/opt/homebrew`) broke libcsp's optional zmq/yaml linking.

v-bus regression: `SOCK_SEQPACKET`

A later commit changed the v-bus socket type from `SOCK_STREAM` to `SOCK_SEQPACKET`, which macOS's kernel does not implement for `AF_UNIX` at all (`socket()` fails immediately with `EPROTONOSUPPORT`). Neither OBC nor ADCS checked `v_bus_initialize()`'s return value, so both processes carried on with a permanently dead `bus_fd` — OBC spun silently in a 100%-CPU busy loop, ADCS just looked "frozen." Fixed by reverting the socket type and adding error checks + reporting throughout `v_bus.c`.

`CSP_PORT_MAX_BIND` silent rejection

libcsp defaults this to 16. The documented port scheme needs up to 30. `csp_bind()` on port 20 failed with `CSP_ERR_INVALID` — silently, no log anywhere in the call chain — while port 10 worked, which made the asymmetry non-obvious. Raised to 31 in the top-level `CMakeLists.txt`.

`xQueueSend()` from a non-task pthread

The headline bug this session: `command_handler.c`'s CSP receive thread is a raw pthread, not a FreeRTOS task. Calling `xQueueSend()` on it hung internally after the very first command — the data still got through (confirmed via targeted debug instrumentation across five layers: `command_handler.c` → `csp_io.c`'s

`csp_read` → `pthread_queue.c`), but the sending thread itself never returned from the call, so it never accepted another connection again. Fixed with `xQueueSendFromISR()`, the documented mechanism for feeding a FreeRTOS queue from outside task context.

Pre-existing scaffolding bugs

A mismatched include guard (`command_task.h` defined `CONTROL_TASK_H`, silently dropping `control_task.h`'s own declarations when both were included), a function-name typo (`telemetry_task` vs. `telemetry_task`), a wrong include in `housekeeping_task.c`, a stale/wrong declaration in `csp_if_spi.h`, and a deleted `drivers/` directory still referenced by the root `CMakeLists.txt` — all blocked a clean build outright and were fixed as prerequisites.

8 Power — Current Capability & Potential

What is proven, today, end to end

- Two independent OS processes, standing in for two independent boards, exchange addressed, routed, framed protocol packets over a swappable bus medium — and that exchange survives sustained, repeated operation (13 consecutive command/response cycles verified in one run), not just a single lucky pass.
- A command entering at one process reliably activates the correct, specific subset of concurrent tasks in the other process (4 of 6, deliberately excluding one), through a real FreeRTOS scheduler running real inter-task synchronization primitives — and nothing else fires.
- A response is generated independently by the activated subsystem and correctly finds its way back to the originating process, closing the loop without any shared memory between the two — everything crosses the wire.
- All of the above is now provably true after a code change, not just after the code was written: two CTest-registered integration tests exercise the real compiled binaries, and both were shown to actually fail when the underlying bug was reintroduced.

What that actually demonstrates, underneath the specific "move to a position" example: **the hard, easy-to-get-wrong plumbing between a Linux mission computer and an RTOS subsystem controller is done and working.** Addressing, routing, port multiplexing, packet framing, cross-thread/cross-task synchronization, and the physical-medium abstraction are the parts of a flight software stack that are tedious and bug-prone to get right and don't visibly "do" anything mission-specific on their own — and they're the parts that, once solid, everything else builds on without having to be re-solved per feature.

Concretely, this means:

- **Adding a new command is now a known, mechanical pattern**, not a design problem: define a struct in `csp_commands.h`, pick the next port off the already-reserved 11/12/21/22 range, add a receive thread and a queue send in the relevant task file, add the send-side call in whichever process originates it. The EPS and COMMS command/telemetry ports are already reserved and waiting for exactly this.
- **The RTOS task skeleton is a template**, not a one-off: six tasks at distinct priorities and periods, with a proven pattern for both event-driven (Command, on a queue) and periodic (everything else, on `xTaskDelayUntil`) tasks, plus a proven non-blocking way for a periodic task to react to an out-of-band event without giving up its schedule.
- **Swapping simulation for real hardware is now an interface problem, not a rewrite**: one CMake flag, one file (`spi_driver.c`), zero changes anywhere in CSP, task, or application code.
- **The regression bar is real**: both integration tests run in seconds, exercise the actual compiled artifacts (not mocks), and are demonstrated to catch the exact class of bug found here — silent hangs and silent packet drops that a manual smoke test would not reliably catch.

9 Next Steps

Highest-leverage next step: OBC services (Phase 8 of the curriculum in AGENTS.md)

OBC's `main.c` is currently hand-rolled send/receive logic with no ground link at all. The documented design (`docs/api_contracts.md`, `docs/data_flow.md`) already specifies four services that don't exist as code yet:

- **CI (Command Ingest)** — UDP listener for ground commands, CRC validation, routing to the Time-Tagged Scheduler.
- **TTS (Time-Tagged Scheduler)** — min-heap by UTC timestamp, 1Hz dispatch loop. This is what would replace the current hardcoded 30-second loop with real scheduled commanding.
- **LC (Limit Checker)** — evaluates telemetry against a rule table, autonomously injects safety commands (e.g. low-battery → safe mode).
- **TO (Telemetry Output)** — formats MCU telemetry into CSP downlink packets over UDP to a ground station.

Hardware bring-up

- Vendor the STM32Cube HAL and wire up an ARM cross-compilation toolchain (toolchain file, linker script, startup code) — none of this exists in the repo yet, which is why `HW_MODE=ON` cannot target real silicon today even though it now builds cleanly for the host.
- Fill in `platform/real/spi/spi_driver.c`'s `STM32_HAL_AVAILABLE` branch against real HAL headers, and validate real SPI timing/chip-select behavior on the bench — the interface is ready; the implementation is

not.

- Physical integration test on actual OBC/MCU hardware once both exist, re-running `position_command_test`'s scenario by hand as the first bring-up milestone.

ADCS subsystem logic

- Replace the "log and move on" stub bodies in Control, Estimation, and Sensor with real logic: PID/B-dot/sun-pointing control, a Kalman filter for attitude estimation, and real IMU/magnetometer/sun-sensor drivers (stub headers already exist under `apps/adcs/include/{control,estimation,manager}/`, entirely unimplemented).
- Implement the ADCS Manager's mode state machine described in its own header comment (Boot → Safe → Detumble → Sun Acquisition → Sun Pointing → Earth Pointing → Science).

Protocol & reliability hardening

- Turn on CSP's HMAC/CRC options with real keys for anything safety-critical, rather than relying on the transport being trustworthy by construction (true in SIM over a local socket; not true over an RF ground link).
- Decide which commands need CSP's RDP (reliable, ordered) mode versus the current fire-and-forget connectionless sends — a safe-mode command dropped silently is a different risk than a telemetry sample dropped silently.
- Extend past the current single-interface-per-node topology (routing table, multiple interfaces) once more than two nodes exist on the bus.

Process & scale

- Wire the existing CTest suite into CI so regressions are caught on every change, not just when someone remembers to run it locally.
- Add EPS (port 11/21) and COMMS (port 12/22) following the exact pattern documented in §8 — the fastest way to validate that the pattern really does generalize.
- Watchdog/fault handling: `docs/architecture.md` already specifies a heartbeat-based MCU watchdog; nothing implements it yet.

10 Appendix — File Map & Quick Reference

Build & run

```
# Simulation build (default)
cmake -S . -B build -DHW_MODE=OFF
cmake --build build

# Run both sides (separate terminals)
./build/apps/obc/obc_sim
./build/apps/adcs/adcs_sim

# Fast-cadence demo run
OBC_POSITION_PERIOD_SEC=2 ./build/apps/obc/obc_sim

# Tests
ctest --test-dir build --output-on-failure

# Hardware-mode build (host-buildable stub; not yet real-hardware-buildable)
cmake -S . -B build-hw -DHW_MODE=ON
cmake --build build-hw
```

Key files by layer

Layer	Path
OBC application	apps/obc/src/main.c
ADCS application	apps/adcs/src/main.c
ADCS tasks	apps/adcs/src/tasks/*.c
ADCS command decode	apps/adcs/src/communication/command_handler.c
FreeRTOS static-alloc hooks	apps/adcs/src/freertos_hooks.c
CSP wiring	shared/csp/csp_network.{c,h}
CSP↔v-bus interface driver	shared/csp/csp_if_spi.{c,h}
Shared command/telemetry structs, ports	shared/csp/csp_commands.h
v-bus interface (canonical)	platform/include/v_bus.h
v-bus SIM backend	platform/sim/spi/v_bus.c
v-bus HW backend (stub)	platform/real/spi/spi_driver.c

libcsp (vendored, patched)	libs/libcsp/
Integration tests	tests/test_v_bus.c, tests/test_position_command.c

Flight Software V1 — System Report. Generated from a direct working session on the codebase; every claim about what builds, runs, or passes in this document was independently re-verified before writing.